

QA Report — Tick & Taka (round 4)

Re-verification of the round-3 QA findings, and a fresh audit of the 36 commits made to fix them

Project	Tick & Taka (monorepo: apps/api, apps/mobile, packages/shared)
Date	7 October 2026
Commit	801dd5f on fix/audit-2026-10-06 — “fix(mobile): drop a write the proxy refuses as too large instead of retrying forever”
Baseline	Round 3 at 7e2bfeb (QA score 87; open: QA-301...QA-309, plus QA-022, QA-026 and QA-211 partly fixed); 36 commits since
Auditor role	QA Lead
Mode	Read-only. No source, config, dependency or git state changed. Scratch scripts lived in docs/audit/2026-10-07-r4/_qa_tmp/ and were deleted afterwards.
Companion reports	UX_Report.pdf, Code_Quality_Report.pdf (written separately)

Severity scale used in all three reports: **Critical** data loss, security hole, crash or broken core flow · **High** major feature broken or a stated principle clearly violated · **Medium** real problem with a workaround, notable friction or debt · **Low** polish, consistency, minor cleanup. Round-4 findings are numbered from QA-401.

1. Executive summary

89 / 100 overall QA score (round 1: 58 · round 2: 70 · round 3: 87)

All nine round-3 findings are fixed, each with tests. Edits behind a stuck create now wait with it, a budget draft touches one line, a pay's Undo is one `/unpay` write, early writes are on disk, a key hiccup keeps the screen cache, and the assistant's settings changes are limited and undoable. Lint, typecheck and 774 tests pass. Of the 12 earlier items, 10 are fixed and 2 (reminders, coverage tooling) stay partly fixed and accepted.

The new code has two Medium problems. **QA-401** (reproduced): a stuck *edit* of an existing account, category, area or budget line takes every later expense or task that uses that record with it. The card reads “Couldn't save, and 2 later changes to it”, and one Discard drops them all. **QA-402** (unverified): the export file is deleted as soon as the share promise resolves, which on Android can be before Drive or Gmail has read it. Delete-account asks for this export just before deleting. Four Low findings follow. No Critical or High issue was found. Device-only behaviour couldn't be run (section 2).

Severity	New in round 4
Critical	0
High	0
Medium	2
Low	4
Total	6

Previous findings (12)	Count
Fixed	10
Partially fixed	2
Not fixed	0
Regressed	0

9 round-3 findings, plus 3 items round 3 left partly fixed

Top 5 to fix first

Sev.	ID	Issue
Medium	QA-401	A stuck edit of an account, category, area or budget line takes every later expense or task that uses that record aside with it, and Discard drops them all. Reproduced
Medium	QA-402	The export is deleted as soon as the share promise resolves, which may be before Drive or Gmail has read it. This is the export the delete-account page asks for. Unverified
Low	QA-403	Undo of a Skip that changed nothing moves the bill back, while that period's payment stays logged. Reproduced
Low	QA-404	A bill paid from a Tiki draft has no Undo, although the draft already holds everything <code>/unpay</code> needs.
Low	QA-405	An Undo on the widget is lost if the app picked up the log while its storage key couldn't be read. Unverified

2. Scope & method

What was reviewed

- **Intent:** `PRODUCT.md`, `DESIGN.md`, `README.md` and the spec (via `_raw/spec.txt`). The round-3 requirement checklist (Appendix C) was carried over and re-checked in section 4.
- **Re-verification:** each of QA-301...QA-309, and QA-022, QA-026 and QA-211, which round 3 left partly fixed. This was done by reading the code at `801dd5f`, reading the new tests, and re-running a reproduction wherever a script could reach the path.
- **Fresh audit** of `git log 7e2bfeb..HEAD` (36 commits). It covered the server-clock guard and pre-2024 edits, stuck-write groups with `Retry`, `Discard` and `Undo`, `outbox.cancel` and `POST /recurring/:id/unpay`, Today's exact `dueAt`, the side key for pre-load writes, save-failure surfacing, the widget hand-over, the encrypted cache on key failure, export clean-up, the error-envelope check and cache buster, `PUT /budgets/:month/:categoryId`, the `update_settings` allow-list and `Undo`, session-touch throttling, the quick-add split, the widget layout and the time-zone hooks.

Commands and evidence

Command	Result	Source
<code>bunx biome check .</code>	555 files, exit 0	<code>_raw/lint.txt</code>
<code>bun run typecheck</code>	shared / api / mobile, exit 0	<code>_raw/typecheck.txt</code>
<code>bun run test</code>	774 pass, 0 fail (shared 247, mobile 291, api 236)	<code>_raw/test.txt</code>
coverage per workspace	api 94.50% lines, shared 95.08%, mobile 93.05% (pure modules only)	<code>_raw/coverage-*.txt</code>
<code>bun run audit (scripts/audit.ts)</code>	5 accepted advisories, none new, exit 0	<code>_raw/audit.txt</code>
<code>bunx expo export -platform android</code>	bundles (2 855 modules)	<code>_raw/android-export.txt</code>
QA probes (Bun): <code>outbox-probe.ts</code> , <code>api-probe.ts</code> , <code>auth-probe.ts</code> , <code>qa-diff.ts</code>	The probes import the project's own code. They use the real mobile <code>OutboxQueue</code> , and <code>createTestContext</code> with in-memory SQLite, an injected clock and real routes. <code>qa-diff.ts</code> compares the quick-add parser with the round-3 parser, taken out with <code>git archive 7e2bfeb</code> . Output is in Appendix B.	<code>_qa_tmpl/ (deleted)</code>

What could not be verified (marked Unverified where it matters)

- **Anything that needs a device.** Sign-in is Google-only, and the only emulator image has no Play services. So none of the following ran on a phone: the widget (its new layout, its quick-logs at the default size, its headless task and the hand-over timing), Android Keystore and `expo-secure-store` failures (the side key, the kept screen cache, QA-405), the share sheet's timing (QA-402), the app lock, notifications and quiet hours, how Hermes' `Intl` handles non-default time zones, and the snackbar's Undo timing. These were judged by reading the code. For QA-402, that included the `expo-sharing` 57.0.22 Android module.
- **Network timing.** A lost reply was simulated by sending the same request twice, which is what the outbox does after a timeout. A persistent 5xx was simulated with a fake `send`.
- **AI.** No real OpenAI calls were made. The `update_settings` allow-list was judged from its code and its tests (`assistant-actions.test.ts:L123-L176`).
- The screenshots in `docs/screenshots` predate every fix and were not used as evidence.

3. Previous findings status

Line references are at 801dd5f . “Re-run” means a probe in this round repeated the round-3 scenario and now gets the expected result (Appendix B).

ID	Sev.	Round-3 title (short)	Status	Evidence at 801dd5f
QA-301	M	Edits queued behind a stuck create are dropped; Retry brings back the old version	Fixed	When a write is set aside, every later write that names a record it creates or edits goes with it (<code>outbox-queue.ts:L424-L443</code>). A write queued later joins the same group (<code>L266-L271</code> , <code>L446-L456</code>). Retry, Discard and “Retry all” act on whole groups (<code>L117-L146</code> , <code>sync-status.tsx:L42-L94</code>). Tested in <code>outbox-queue.test.ts:L327-L400</code> . Re-run: the PATCH is set aside with its POST, and after Retry the title is “Call bank about card”. Residual: a user DB that won't open still answers 500, not 503 (<code>user-registry.ts:L184-L192</code>), so an outage sets every write aside after about 2 min; “Retry all” makes that a single tap. The grouping rule now goes too far: see QA-401.
QA-302	M	An old budget draft or its Undo overwrites the whole month	Fixed	New <code>PUT /budgets/:month/:categoryId</code> (<code>budgets/routes.ts</code> , <code>service.ts:L291-L343</code>) sets one line, or removes it with <code>null</code> . It keeps rollover when rollover isn't sent, and copies an inherited month's lines in first. <code>set_budget</code> , its draft and its Undo use it (<code>actions.ts:L178-L228</code>). Tested in <code>assistant-money-drafts.test.ts:L152-L262</code> . Re-run in an inheriting month: Food set, Fun kept at ₺2,000 with rollover. Then Fun set to ₺3,000 and Food's Undo sent: Fun stays at ₺3,000.
QA-303	L	Undo after a replayed pay leaves the due date a month ahead	Fixed	A replay names <code>previousDueAt</code> while the bill sits where that pay put it (<code>recurring/service.ts:L120-L132</code>). Undo is now one write, <code>POST /recurring/:id/unpay {transactionId, dueAt, skip}</code> (<code>L193-L209</code>), queued without waiting for the pay's reply (<code>recurring-pay.ts:L36-L52</code>). Today's bills carry <code>dueAt</code> (<code>today/service.ts</code>), and the cache buster was bumped to 3. Tested in <code>recurring.test.ts:L109-L232</code> . Re-run with an overdue bill: Today <code>at</code> = 4 Oct, <code>dueAt</code> = 1 Oct, and pay then unpay gives <code>nextDueAt</code> = 1 Oct. New edge with Skip: QA-403.

ID	Sev.	Round-3 title (short)	Status	Evidence at 801dd5f
QA-304	L	Writes queued before the outbox loads exist only in memory	Fixed	Before load, writes go to <code>tt.outbox.early</code> and are merged by <code>load()</code> (<code>outbox-queue.ts:L199-L258</code> , <code>L493-L512</code> , <code>outbox.ts:L22-L44</code>). The widget hand-over awaits <code>startOutbox()</code> and trims its own list only when <code>durable()</code> is true (<code>widget-pending.ts:L32-L46</code> , <code>outbox-queue.ts:L343-L349</code>). The status line says “not saved on this phone yet” (<code>sync-status.tsx:L113-L114</code>). Tested in <code>outbox-durability.test.ts</code> and <code>widget-handover.test.ts</code> . Residual: QA-405.
QA-305	L	A key hiccup deletes the encrypted screen cache	Fixed	<code>keepCacheOnKeyFailure</code> turns <code>KeyUnavailableError</code> into “no cache” and won't save over the sealed copy until it has been read. It then merges that copy late (<code>cache-persister.ts:L14-L57</code> , <code>query-client.ts:L83-L97</code>). The cache store refuses a plain-text fallback (<code>secure-storage.ts:L67-L76</code>). Tested in <code>cache-persister.test.ts:L102-L150</code> .
QA-306	L	Assistant changes any setting with no Undo	Fixed	An allow-list leaves out <code>appLock</code> , <code>lockAfterMinutes</code> , <code>ai</code> , <code>defaultCurrency</code> , the default and cash accounts, and <code>timeZone</code> (<code>actions.ts:L27-L48</code>). Each change records an Undo with the old values, and the vacation list comes back with <code>vacationMode</code> (<code>L243-L264</code>). Tested in <code>assistant-actions.test.ts:L123-L176</code> , including <code>{theme:"dark", appLock:false}</code> , which is refused as a whole.
QA-307	L	Export file left in the cache after sign-out	Fixed	The file is deleted in <code>finally</code> after <code>shareAsync</code> (<code>use-export-data.ts:L26-L34</code>), and any leftover export is swept in <code>clearUserData</code> (<code>user-data.ts:L19-L26</code> , <code>L40</code>). Tested in <code>export-file.test.ts</code> . Not done: the export is still parsed into memory and stringified again (<code>L16-L19</code>). The immediate delete brings a new risk, QA-402. That risk partly comes from round 3's own recommendation (“delete the file once <code>shareAsync</code> resolves”).
QA-308	L	Ten client spots use Asia/Dhaka for today and this month	Fixed	<code>useToday</code> , <code>useThisMonth</code> and <code>useUserTime</code> are in <code>lib/use-today.ts</code> . All ten spots now use them (checked one by one). <code>time-zone-use.test.ts:L91</code> scans every screen for a <code>toLocaleDate / toLocalMonth</code> call without a zone, which is the CI guard round 3 asked for. A grep for one-argument calls finds none.

ID	Sev.	Round-3 title (short)	Status	Evidence at 801dd5f
QA-309	L	The glue around the fixes is untested	Fixed	New tests: <code>sync-pull.test.ts</code> (pull, cursor, waiting for the outbox), <code>outbox-failure.test.ts</code> (proxy and captive-portal mapping, 413), <code>widget-handover.test.ts</code> , <code>turn-recovery.test.ts</code> , <code>cache-persister.test.ts</code> , multi-entry parking, the replayed pay, and the other-line budget case. Two small items remain untested: the <code>storageKey</code> retry in <code>secure-storage.ts</code> , and the <code>applyOptimistic</code> wrapper. Both are folded into QA-406.
QA-211	M (r2)	New failure paths untested	Fixed	The part round 3 left open, the sync pull, is now covered by <code>sync-pull.test.ts</code> .
QA-022	M (r1)	Reminder gaps	Partially fixed	No change since round 2. Reminders are rescheduled only when the app opens, with a 7-day horizon (<code>plan-notifications.ts:L37</code>). There is no background task (no <code>expo-task-manager</code> in <code>package.json</code>). Accepted as Low; not raised again.
QA-026	L (r1)	Coverage numbers unreliable	Partially fixed	Documented in <code>README.md:L393-L397</code> . The cross-workspace runs still show <code>shared/src/dates.ts</code> at 5.86% (api) and 0.00% (mobile) lines, and the mobile run shows <code>server-clock.ts</code> at 63.89% even though <code>server-clock.test.ts</code> exercises the uncovered lines. Accepted as a Bun tooling limit.

Tally: 10 fixed, 2 partially fixed, 0 not fixed, 0 regressed. All 9 round-3 findings are fixed.

4. Requirements traceability

Requirements come from the spec's build checklist and technical rules, `PRODUCT.md`, and the fix contracts adopted in rounds 1 to 3 (Appendix C). Status: **Implemented** **Partial** **Missing** **Broken**. "Implemented" means the code path exists and is tested or was read end to end. It does not mean it was run on a device.

Requirement	Implementation	Status	Notes
Sign-in, sessions, per-user SQLite + migrations	auth/routes.ts, middleware/auth.ts, db/sessions.ts, db/client.ts	Implemented	<code>last_used_at</code> is written at most once a minute, and the grace-window shortening only once (<code>sessions.ts:L110-L119</code>). Nothing reads <code>lastUsedAt</code> , so the throttle changes no behaviour.
Auth on every protected route	app.ts, requireAuth	Implemented	The new routes are under <code>requireAuth</code> . Re-run: <code>POST /recurring/:id/unpay</code> and <code>PUT /budgets/:month/:categoryId</code> without a token both return 401.
Offline cache + queued writes in order	outbox-*.ts, query-client.ts, secure-storage.ts, cache-persister.ts	Partial	Durable from the first tap, and stuck writes are grouped and recoverable. The grouping catches writes that don't depend on the stuck one (QA-401).
Conflicts: last write wins on <code>updated_at</code>	crud.ts:L83-L97, server-clock.ts	Implemented	An offset is learned only from a real <code>x-server-time</code> on or after 2024. An edit stamped before 2024 applies in arrival order. Re-run: a 1970 stamp applies, and a 2025 stamp older than the row loses.
GET <code>/sync/changes</code> and the pull	sync/routes.ts, sync-pull.ts	Implemented	The pull waits for the outbox to empty, asks for counts only, and advances the cursor only after a good reply. Now tested.
Quick-add (Bangla + English)	shared/quick-add*.ts, digits.ts, calculator.ts	Implemented	The split into money, task and guess modules changes nothing. 230 cases (46 inputs × 5 forced kinds) give identical results to <code>7e2bfeb</code> (Appendix B).
Tasks, subtasks, repeat, undo; planner moves	tasks/*, plan/*	Implemented	—
Recurrence in plain words	shared/recurrence.ts	Implemented	—
Focus timer / time tracking	time-entries/service.ts, focus-screen.tsx	Implemented	Uses server time; a header-less reply can no longer push it to 1970.
Habits, streaks, freezes; routines	habits/*, routines/*	Implemented	The default habit date follows the zone setting (<code>habits/queries.ts:L7</code>).
Accounts, computed balances, balance check	accounts/service.ts	Implemented	A stuck balance check sets aside later expenses on that account (QA-401).

Requirement	Implementation	Status	Notes
Budgets: buckets, rollover; safe to spend	budgets/service.ts, shared/finance.ts	Implemented	The one-line PUT keeps the other lines and rollover. Re-run in Appendix B.
Bills: "Paid" creates a transaction; Undo	recurring/service.ts, money/recurring-pay.ts	Implemented	Undo is server-side and survives a restart. Edges: QA-403 (Skip) and QA-404 (Tiki).
Goals, debts, keypad, events, shopping	goals/, debts/, calculator.ts, events/, shopping/	Implemented	Dates are picked through the new <code>dateField</code> , in the user's zone.
SMS auto-capture	—	Missing	Dropped by a recorded decision.
Widget, launcher shortcuts	features/widget, use-app-shortcuts.ts	Implemented	Unverified At the default size, quick-logs replace Focus and Tiki, and every pill is at least 48 dp (<code>widget-layout.ts</code> , tested). Hand-over: QA-405.
Local notifications, quiet hours, bill reminder	features/notifications/*	Partial	Rescheduled only when the app opens (QA-022 residual).
Nightly backup (keep 14), off-site copy, JSON export	jobs/backup.ts, jobs/offsite.ts, export route, use-export-data.ts	Implemented	The export file is deleted right after sharing. The timing may break the share (QA-402, Unverified).
Fingerprint lock after 5 min; privacy mode	app-lock.tsx, lock-decision.ts	Implemented	Unverified The assistant can no longer turn it off.
Data at rest protected	encrypted-storage.ts, secure-storage.ts	Implemented	Key failures no longer cost the outbox or the cache. The widget store is the documented exception.
Login 5 / 15 min; API on 127.0.0.1	auth/routes.ts, env.ts	Implemented	—
AI with graceful fallback; cost cap	ai/usage.ts, modules/ai/*	Implemented	—
AI never saves money without confirmation	assistant/tools.ts, actions.ts	Implemented	Settings are allow-listed and come with an Undo.
Undo instead of "Are you sure?"	notify + outbox writes	Partial	It holds almost everywhere. Gaps: a Tiki-paid bill (QA-404), and a Discard whose group is larger than the card says (QA-401).
Account deletion	DELETE /auth/account, settings/delete-account.tsx	Implemented	"Export first" depends on the share working (QA-402).

5. Findings

Grouped by category, with categories ordered by their worst finding. **Reproduced** means a Bun script calling the project's own code produced the output shown; full output is in Appendix B. **Unverified** means device behaviour judged by reading the code.

5.1 Offline sync

QA-401 · A stuck edit of an account, category, area or budget line takes every later write that uses that record aside with it, and Discard drops them all

Medium

Category: Offline / data loss Effort: S-M

Reproduced

Where: `apps/mobile/src/lib/outbox-policy.ts:L110-L136` (`writtenIds` takes every record id in the path; `mentionsAny` matches any id anywhere in the body). `outbox-queue.ts:L424-L443` (`park`), `:L266-L271` and `:L446-L456` (a new write joins a group), `:L138-L146` (`discardStuck` drops the whole group). `components/ui/sync-status.tsx:L19-L26, L55-L58, L72-L78` (card and Discard). The labels used as the description: `money/account-sheet.tsx:L62-L66`, `settings/category-sheet.tsx:L73`, `plan/area-sheet.tsx:L61` ("Couldn't save").

What's wrong: The QA-301 fix sets aside "every later write that names a record it creates or edits". For a stuck *create* that's right. For a stuck *edit* of a record that already exists, it isn't. `PATCH /accounts/A` (rename), `POST /accounts/A/balance-check`, `PATCH /categories/C`, `PATCH /areas/X` and `PUT /budgets/2026-10/C` (Tiki's budget draft) all put an existing record's id in the path. After that, any expense with `accountId: A` or `categoryId: C`, and any task with `areaId: X`, joins the stuck group, even though it would land without trouble. Every write queued later also joins at once, and its promise rejects with `SetAsideError`, while the screen has already shown it as saved. The card describes the group by the head's `label`, which is failure text. So it reads "Couldn't save, and 2 later changes to it". One Discard then removes the account edit and every expense in the group. Only an 8-second snackbar Undo protects them.

```
// Real OutboxQueue; PATCH /accounts/A answers 500 every time, everything else works.
OVERGROUP: stuck = ["PATCH /accounts/01M4...", "POST /transactions", "POST /transactions"]
    | T2 promise: SetAsideError | expenses on server: 0
OVERGROUP: Discard on the account edit dropped 3 writes | expenses on server: 0 | queue
size 0
BUDGET-GROUP: stuck = ["PUT /budgets/2026-10/01M4...", "POST /transactions"] | expense on
server: false
```

Why it matters: The trigger is uncommon: one edit has to fail 8 times with a non-503 5xx. When it happens, though, the result works against the product's core numbers. Expenses the user logged aren't on the server. So after the next refetch they disappear from Today's safe-to-spend, and one tap on Discard, made to drop a failed rename, deletes them for good. The outbox tests only check writes that depend on a stuck create (`outbox-queue.test.ts:L327-L400`). Nothing checks that unrelated users of an edited record keep going.

Recommendation: Only a *create* passes its dependency on to writes that mention it in the body. For an edit, a delete or an action, group only later writes whose *path* targets the same record. Describe the group by kind ("Edit account bKash · 2 expenses waiting with it"), not by the failure label. Before discarding a group that contains creates, name them in the snackbar.

```
const created = new Set(createdIds(entry.request)); // POST body.id / transactionId
const targets = new Set(pathIds(entry.request)); // record an edit/delete/action
acts on
const dependsOn = (later: OutboxRequest) =>
  mentionsAny(later, created) || pathIds(later).some((id) => targets.has(id) ||
  created.has(id));
```

5.2 Privacy & data portability

QA-402 · The export file is deleted as soon as the share promise resolves, which may be before the chosen app has read it

Medium

Category: Data portability / account deletion Effort: S

Unverified on device

Where: apps/mobile/src/features/settings/use-export-data.ts:L16-L34 .

features/settings/delete-account.tsx:L41-L53 ("1. Keep a copy ... Save it before you go on").
node_modules/expo-sharing/android/src/main/java/expo/modules/sharing/SharingModule.kt:L39-L52, L80-L83 (the chooser is started for result, and the promise resolves on any activity result).

What's wrong: The QA-307 fix deletes the file in `finally`, right after `await Sharing.shareAsync(file.uri)`. On Android that promise resolves when the chooser returns an activity result. Some targets return it as soon as they open, especially ones started in their own task. Others return it when their own screen closes, then upload or attach the file afterwards from the `content:// URI` (Drive's background upload, a mail draft sent later). In both cases the file may already be gone when the target reads it. The upload fails, or the attachment is empty, and the app has already shown nothing went wrong.

```
const data = await send("GET", "/export");
file = new File(Paths.cache, exportFileName(new Date()));
file.write(JSON.stringify(data));
await Sharing.shareAsync(file.uri, { mimeType: "application/json", ... });
} finally {
  if (file?.exists) file.delete(); // runs as soon as the chooser reports back
```

Why it matters: The export is the user's only copy of their data outside the server. The delete-account page asks for it as step 1, just before an irreversible delete. If the share silently failed, the user deletes the account believing they have a backup. Round 3's recommendation ("Delete the file once `shareAsync` resolves") didn't account for this. This report corrects it.

Recommendation: Don't delete right after the share. Sweep exports older than about an hour at the next app start, and on sign-out (already done). Better: save through the Storage Access Framework (`Directory.pickDirectoryAsync()` in expo-file-system), so the app writes the file to a folder the user picks, knows it was written, and can say "Saved to Downloads/tick-taka-export-2026-10-07.json". Then delete-account can show that confirmation. Test on a device with Drive, Gmail and Files as targets.

5.3 Money & Undo

QA-403 · Undo of a Skip that changed nothing moves the bill back, while that period's payment stays logged

Low

Category: Money / Undo Effort: S

Reproduced

Where: `apps/api/src/modules/recurring/service.ts:L71-L78` (`movedOnceFrom` infers the move from dates), `:L133-L134` (a late pay or skip changes nothing), `:L201-L206` (`unpay`: “A skip logs nothing, so only the date says whether it went through”).

What's wrong: A skip leaves no record. So `unpay {skip:true}` moves the bill back whenever its date is one step past `dueAt`, whoever moved it there. Here is how that happens. The user taps Paid on Today (offline, or while the recurring sheet is still cached). They then open the bill's sheet, which still shows 5 Oct, tap Skip, and then Undo. The Skip did nothing, because the bill had already moved. But its Undo moves the bill back to 5 Oct, and October's expense stays logged.

```
STALE-SKIP-UNDO: skip reply moved=false | unpay movedBack=true nextDueAt=2026-10-05
                | October expenses still logged=1
```

Why it matters: The bill shows as due, then overdue, for a period that is already paid, which invites a second payment. The trigger is narrow, since it needs a stale screen and an Undo.

Recommendation: Record who moved the bill. Store the pay's or skip's `transactionId` on the recurring row when it advances (for example `moved_by`), and have `unpay` move it back only when `moved_by === input.transactionId`. This replaces the date inference for both pay and skip. Add the test from section 8.

QA-404 · A bill paid from a Tiki draft has no Undo, although the draft holds everything /unpay needs

Low

Category: AI / Undo Effort: S

Where: `apps/api/src/modules/ai/assistant/actions.ts:L92-L120` (`pay_bill` proposes and records with no `undo`). `apps/mobile/src/features/assistant/use-assistant.ts:L220-L240` (Save records `undo: draft.undo`, which is undefined here).

`apps/api/src/modules/transactions/service.ts` (deleting the expense doesn't touch the bill). `README.md` (“money actions ... are corrected in their own screens”).

What's wrong: The app always asks for drafts (`draftMoney: true`), so “pay my internet bill” becomes a pay draft with `transactionId` and `dueAt`. After Save, the chat shows “Saved: Pay Internet” with no Undo. The README's suggested fix, deleting the expense in Money, leaves the bill a month ahead. That is the same symptom QA-303 fixed for Today. Contributions to goals and debt repayments in the chat also have no Undo, but their reversal isn't a single write.

Why it matters: Today's Paid can be undone, but the same pay from Tiki can't. The user has to know to edit the due date by hand. This goes against the app's own rule, “Undo instead of ‘Are you sure?’, as everywhere else in the app” (`sync-status.tsx:L73`).

Recommendation: In `pay_bill`, add `undo: { method: "POST", path: `${billPath}/unpay`, body: { transactionId, dueAt, skip: Boolean(raw.skip) } }` to both the draft and the direct

action, and update the README line.

5.4 Widget

QA-405 · An Undo on the widget is lost if the app picked up the log while its storage key couldn't be read

Low

Category: Widget / offline

Effort: S

Unverified on device

Where: `apps/mobile/src/features/widget/widget-pending.ts:L32-L46` (logs are queued, then kept on the widget when `durable()` is false). `widget-task-handler.tsx:L64-L66` (Undo only removes the log from the widget's list). `lib/outbox-queue.ts:L209-L216, L499-L511` (while the saved queue can't be read, writes wait on the side key and are sent once it can be read).

What's wrong: When the outbox can't read its saved queue (a Keystore hiccup), the hand-over still queues each widget log. The log goes to the outbox's side key, saved in plain text by the fallback. The widget keeps its own copy, which is correct for durability. If the user then taps Undo on the widget within its 10-minute window, `undoQuickLog` finds the log in the widget's list, removes it from there, and queues no delete. The outbox's copy is still on the side key. It is sent once the key reads again, and the expense the user took back is saved.

Why it matters: Today's spending is too high by the amount taken back, and nothing on screen says why. It needs a Keystore failure and a widget Undo, so it is rare.

Recommendation: When `undoQuickLog` removes a log from the waiting list, also add its id to the pending deletes. A `DELETE` for an id the server never saw answers 404, which the outbox already treats as done (`outbox-policy.ts:L70-L73`). If the outbox did send the create, the delete follows it in order.

5.5 Testing

QA-406 · The new rules are tested on their happy paths; the edges behind QA-401, QA-403 and QA-405 and a few modules are not

Low

Category: Testing

Effort: M

Where: `apps/mobile/test/outbox-queue.test.ts:L297-L400` (parking groups: creates only). `apps/api/test/recurring.test.ts:L146-L232` (no unpay for a skip that changed nothing). `widget-handover.test.ts` (no widget Undo during the hand-over). `secure-storage.ts:L13-L25` (`storageKey` retry: no test). `lib/outbox.ts:L86-L108` (`applyOptimistic` wrapper). `packages/shared/src/quick-add.ts:L125-L134, L151-L184` (forced time entry and `describeDraft`: 52.69% lines, `_raw/coverage-shared.txt`). `use-export-data.ts` (no test; QA-402 needs a device test).

Quality of what exists: Good. There are 774 tests, up from 689. Every round-3 finding has a regression test that names it. The new tests use the real queue, real routes and real `Headers`, not stand-ins (`server-clock.test.ts`). A source scan guards the time-zone rule. No flaky patterns were seen; the outbox tests poll with short bounded waits.

Recommendation: Add the cases in section 8.

6. Strengths (keep these)

- **Every fix ships with its proof.** Each of the 36 commits names the finding it closes. The fix commits add a test that fails without the fix: `outbox-durability.test.ts`, `widget-handover.test.ts`, `cache-persister.test.ts`, the `unpay` suite and the other-line budget suite. The quick-add split preserved behaviour exactly (230 of 230 cases).
- **Undo moved to the server, where it belongs.** `/unpay` needs nothing from the pay's reply, is safe to repeat, refuses another bill's transaction, and keeps a later period's date. A pay that hasn't gone out is simply taken off the queue (`outbox.cancel`).
- **The outbox is honest about storage.** Writes are on disk from the first tap. A failed save is reported once and shown until a save works. The widget keeps its own copy until the outbox is durable. A sealed queue that can't be read is never written over.
- **Defensive time handling.** A missing `x-server-time` is NaN, not 1970. A saved offset that puts "now" before 2024 is ignored, and the server treats a pre-2024 stamp as unstamped. Every screen uses the user's zone, and a test enforces it.
- **Narrow assistant powers.** Settings are allow-listed with an Undo, and budget drafts touch one line. A refused key refuses the whole call, so a mixed request can't sneak a risky key through.
- **Failure-shape awareness.** The error-envelope check tells proxy pages apart from API refusals. A 413 is dropped instead of retried forever. The cache buster is versioned, with a comment that says when to bump it.
- **Clean gates.** Biome, typecheck, 774 tests and the audit allow-list all pass in CI, and the README explains the coverage limits.

7. Action plan

Order	ID	Title	Severity	Effort
1	QA-401	Group only true dependants of a stuck write; describe groups by kind; name the creates a Discard drops	Medium	S-M
2	QA-402	Keep the export until a later sweep, or save it through SAF; confirm where it was saved before delete-account	Medium	S
3	QA-406	Tests for the cases in section 8 (P1 first)	Low	M
4	QA-403	Record who moved the bill (moved_by) and make <code>unpay</code> check it	Low	S
5	QA-404	Give <code>pay_bill</code> an <code>/unpay</code> Undo; update the README line	Low	S
6	QA-405	Widget Undo of a waiting log also queues a delete	Low	S
7	QA-301 residual	Return 503 when a user's database can't be opened	Low	S

8. Recommended test plan

P1 covers the Medium findings and P2 the Low ones. "API" means `createTestContext`. "Unit (mobile)" means the real `OutboxQueue` with a fake `send`, as in this audit's probes.

Pri	Type	Covers	Given / When / Then
P1	Unit (mobile)	QA-401	Given a queued <code>PATCH /accounts/A</code> that gets 500 eight times and a queued <code>POST /transactions {accountId:A}</code> , when the queue runs, then only the <code>PATCH</code> is stuck, the expense is sent, and a later expense on <code>A</code> goes out at once.
P1	Unit (mobile)	QA-401	Given a stuck <code>PUT /budgets/2026-10/C</code> , when an expense with <code>categoryId:C</code> is queued, then it is sent, not set aside, and its promise resolves.
P1	Unit (mobile)	QA-401	Given a stuck <code>POST /accounts {id:A}</code> (a create), when an expense in <code>A</code> is queued, then it joins the group (the existing behaviour stays).
P1	E2E (device)	QA-402	Given <code>Settings > Delete account</code> , when <code>Export</code> is shared to Drive, then to Gmail, then to Files, then each target receives a complete file (size > 0, valid JSON), checked after the upload or send finishes.
P2	API	QA-403	Given a bill paid for 5 Oct, when a skip for 5 Oct is sent and then unpaid, then <code>movedBack</code> is false, <code>nextDueAt</code> stays 5 Nov, and the expense stays.
P2	API (fake AI)	QA-404	Given a <code>pay_bill</code> draft, then it carries an Undo to <code>/recurring/:id/unpay</code> with the same <code>transactionId</code> and <code>dueAt</code> . When the draft and then its Undo are sent, then no expense remains and the date is back.
P2	Unit (mobile)	QA-405	Given an outbox whose saved queue can't be read and one widget log handed over, when the widget's Undo runs and the key later reads, then the server receives the <code>POST</code> and a <code>DELETE</code> for that id, in that order, or neither.
P2	Unit (mobile)	secure - storage	Given <code>SecureStore.getItemAsync</code> rejecting once, when a value is read twice, then the first read throws <code>KeyUnavailableError</code> and the second succeeds with the same key (no new key generated).
P2	Unit (shared)	quick-add	Given <code>forceKind:"time_entry"</code> with "email", then a 0-minute, low-confidence entry. Given each draft kind, then <code>describeDraft</code> gives the documented one-line preview.
P2	Unit (mobile)	outbox glue	Given a USD-account expense, when <code>queueWrite</code> runs, then today's left-to-spend is unchanged. A BDT expense lowers it by its amount.
P2	API	QA-301 residual	Given a user whose database fails to open, when any authenticated request arrives, then 503, and the phone's policy retries without parking.

9. Manual smoke test (≈15 minutes, real Android phone)

Production APK against a staging API. Steps marked  target findings in this report; a failure there confirms the finding.

#	Step	Expected
1	Cold-start signed in, with the token older than 24 h, five times on mobile data.	Today loads from cache; no login screen.
2	Airplane mode: log “rickshaw 60 bkash”, edit it to 70, then swipe the app away at once. Reopen and reconnect.	One ₳70 expense on the server (side key, QA-304).
3	Tap “Paid” on a bill in Today, then Undo. Repeat in airplane mode, kill the app before reconnecting, then reconnect.	Both times the expense is gone and the bill is back on its old due date (QA-303).
4	 Tap Paid on a bill in Today. While offline, open the same bill's sheet, tap Skip, then Undo. Reconnect.	The bill stays on next month and October's expense stays (QA-403).
5	 Staging: make <code>PATCH /accounts/:id</code> return 500. Rename bKash, then log two expenses from bKash. Wait 3 minutes.	Only the rename is stuck; both expenses reach the server (QA-401).
6	 Settings › Delete account › Export: share to Google Drive, then open the uploaded file on a computer.	A complete JSON file (QA-402).
7	Ask Tiki “set my food budget to 8000”; don't save. In Budgets change Transport. Save the draft, then tap its Undo.	Food changes and then goes back; Transport keeps its new value (QA-302).
8	 Ask Tiki “pay my internet bill” and Save.	An Undo next to “Saved” (QA-404).
9	Ask Tiki “turn off app lock”, then “switch to dark theme”, then tap Undo.	The first is refused with a pointer to Settings; the theme changes and Undo restores it (QA-306).
10	Add the widget at its default size.	Task, Expense and up to two quick-logs fit in one row, each at least a thumb wide.
11	Airplane mode: tap a widget quick-log, open the app, close it, then tap Undo on the widget. Reconnect.	No expense on the server.
12	Settings › Time zone: America/New_York. Open Budgets and Monthly review at 01:00 on the 1st, Dhaka time.	Both open on the previous month (QA-308).
13	Sign out, then with <code>adb shell run-as</code> on a debug build list <code>cache/</code> .	No <code>tick-taka-export-*.json</code> (QA-307).
14	Turn on App lock, swipe the app away, reopen.	Fingerprint prompt before any content.

Appendix A — Raw command output (trimmed)

Lint

```
$ bunx biome check .
Checked 555 files in 374ms. No fixes applied.
exit 0
```

Typecheck

```
@tick-taka/shared typecheck: Exited with code 0
@tick-taka/api typecheck: Exited with code 0
@tick-taka/mobile typecheck: Exited with code 0
```

Tests

```
@tick-taka/shared test: 247 pass 0 fail 519 expect() calls (13 files)
@tick-taka/mobile test: 291 pass 0 fail 865 expect() calls (56 files)
@tick-taka/api test: 236 pass 0 fail 848 expect() calls (31 files) [9.65s]
```

Coverage (selected lines)

```
shared All files 84.16 % funcs | 95.08 % lines
  src/quick-add.ts 75.00 | 52.69 | 125-134,151-184 <- QA-406
  src/quick-add-money.ts 100.00 | 100.00
  src/quick-add-task.ts 100.00 | 98.43 | 64-65
mobile All files 90.97 % funcs | 93.05 % lines (pure modules only)
  src/lib/outbox-queue.ts 91.89 | 96.82 | 171-174,183,187-189,252-253
  src/lib/cache-persister.ts 87.50 | 86.84 | 49,51-54
  src/lib/server-clock.ts 62.50 | 63.89 <- QA-026 (tested; mis-
mapped)
  ../../packages/shared/src/dates.ts 45.16 | 0.00 <- QA-026
api All files 88.20 % funcs | 94.50 % lines
  src/modules/recurring/service.ts 100.00 | 99.36
  src/modules/budgets/service.ts 100.00 | 100.00
  src/modules/ai/assistant/actions.ts 100.00 | 96.63 | 112-118
```

Dependency audit

```
$ bun scripts/audit.ts
accepted GHSA-vfj7-8cjh-p6xm braces (high): build tooling only; no patched release exists.
accepted GHSA-vcc3-ghjq-m6fr decode-uri-component (moderate): under expo-router ...
accepted GHSA-67mh-4wv8-2f99 esbuild (moderate): inside drizzle-kit's loader ...
accepted GHSA-86w9-cpqp-85rv node-forge (high): Expo CLI tooling only ...
accepted GHSA-rj75-hqrm-r3gf postcss-selector-parser (moderate): Tailwind 3 needs 6.x ...
No new advisories (5 accepted). exit 0
```

Android bundle

```
Android Bundled 51854ms apps/mobile/index.ts (2855 modules)
_expo/static/js/android/index-....hbc (7.7MB) exit 0
```

Appendix B — QA reproduction output

The scripts imported the project's own modules: `apps/api/test/helpers.ts` (in-memory SQLite, clock fixed at Sun 4 Oct 2026 10:00 Dhaka), `money-helpers.ts`, the real mobile `OutboxQueue` and outbox policy, and the round-3 shared sources taken out with `git archive 7e2bfeb`. They lived in `docs/audit/2026-10-07-r4/_qa_tmp/` and were deleted after the run. No source file was changed. Long record ids are shortened with "...".

outbox-probe.ts (apps/mobile OutboxQueue, fake send)

```
OVERGROUP: stuck = ["PATCH /accounts/01M4...", "POST /transactions", "POST /transactions"]
| T2 promise: SetAsideError | expenses on server: 0 (QA-401)
OVERGROUP: Discard on the account edit dropped 3 writes: ["PATCH /accounts/01M4...",
"POST /transactions", "POST /transactions"] | expenses on server: 0 | queue size 0
BUDGET-GROUP: stuck = ["PUT /budgets/2026-10/01M4...", "POST /transactions"] | expense on
server: false
QA-301: stuck = ["POST", "PATCH"] rejected = []
QA-301: after Retry title = "Call bank about card" | queue size 0 (fixed)
```

api-probe.ts · auth-probe.ts (apps/api)

```
STALE-SKIP-UNDO: skip reply moved=false | unpaid movedBack=true nextDueAt=2026-10-05
| October expenses still logged=1 (QA-403)
BUDGET-LINE: unknown category -> 400 {"error":{"code":"invalid_reference",...}}
| income category -> 200 | month 2026-13 -> 400 | limit -1 -> 400
BUDGET-INHERIT: after Food line in inheriting Oct, Fun kept=200000 rollover=true
| after Undo Food=600000 Fun=300000 (QA-302 fixed)
PRE-2024: 1970-stamped -> "Call bank (1970)" | 2020-stamped -> "Call bank (2020)"
| 2025-stamped (older than row) -> "Call bank (2020)"
TODAY-DUEAT: overdue bill at=2026-10-04 dueAt=2026-10-01
| pay+unpay -> nextDueAt=2026-10-01 movedBack=true (QA-303 fixed)
AUTH: POST /recurring/:id/unpay no token -> 401 | PUT /budgets/:month/:cat no token -> 401
```

qa-diff.ts (packages/shared, parser at 7e2bfeb vs 801dd5f)

```
quick-add split: 230 cases (46 inputs × 5 kinds), 0 differences vs 7e2bfeb
"dinner 1,50,000" -> expense 15000000 high "500tk dinner" -> expense 50000 high
"bus 30 + rickshaw 40" -> expense 7000 low "tea 10 tea 10 tea 10" -> expense 1000 low
"dinner 1.5k" -> task "Dinner 1.5k" low "1e3 lunch" -> task
Inputs included: চা ২০, সিঙ্গারা ১০ · চা ২০। · lunch 2x150 · lunch ১,২৫০ · রিকশা ৬০ টাকা ·
১০০০ বেতন · +১০০০ salary · ৩০ মিনিট research · gym every mon wed fri 7am · someday learn
guitar ...
```

Appendix C — Requirement checklist used

From PRODUCT.md and the spec: log anything in under 5 s. Today answers “what now?” and “can I afford it?”. Safe to spend = flexible left ÷ days left including today, with fixed bills excluded. Amounts are integer minor units. Offline cache with queued writes sent in order, ids made on the phone, last-write-wins on `updated_at`. Asia/Dhaka by default for display, with local strings for habit dates and budget months. Streaks with 2 freezes a month. Reward, never guilt. Honest numbers (balances and hours reconcile). AI: rules first, a cost cap, graceful fallback, minimum data, an off switch, and never saving money without confirmation. App lock after 5 min. Receipts served only to authenticated requests. Login limited to 5 per 15 min. Bun listens on 127.0.0.1. Nightly `VACUUM INTO` backup keeping 14. JSON export. Local notifications with quiet hours 23:00–07:00. Bangla and English quick-add.

The fix contracts from rounds 1 to 3 add: revocable sliding sessions with a 60 s grace for a refreshed-away token. A 401 never clears the queue, and signing out with pending writes asks first. Zone validation and per-user job isolation. CHECK → 400. Stuck-write parking with Retry and Discard. Money drafts for budgets and amount fields. A persisted clock offset. Round 3 adds: dependent writes grouped with a stuck one, one-line budget writes, an Undo of a pay that survives a lost reply, writes durable before the outbox loads, no cache loss on a key failure, assistant settings limited with an Undo, export clean-up, and zone-aware today/month on every screen.

Appendix D — File inventory reviewed

Area	Files
Shared	packages/shared/src/{dates, ids, quick-add, quick-add-money, quick-add-task, quick-add-guess}.ts; schemas/{common, money, settings}.ts
API	app.ts; lib/crud.ts; db/{sessions, user-registry, seed}.ts; modules/recurring/{routes, service}; modules/budgets/{routes, service}; modules/today/service; modules/settings/{routes, service}; modules/transactions/service; modules/ai/assistant/{actions, tools, entities}; middleware/error-handler
Mobile lib	outbox-queue, outbox-policy, outbox-failure, outbox, server-clock, http, api, cache-persister, query-client, encrypted-storage, secure-storage, user-data, export-file, sync-pull, sync-changes, use-today, snack-timing, pick-date, format
Mobile features	components/ui/{sync-status, date-field}; widget/{use-widget-sync, widget-pending, widget-task-handler, widget-quick-log, widget-layout}; money/{recurring-pay, recurring-sheet, account-sheet, goal-sheet}; today/next-up; assistant/{use-assistant, money-drafts}; settings/{use-export-data, delete-account, category-sheet}; plan/area-sheet; the ten QA-308 screens
Tests	apps/mobile/test/{outbox-queue, outbox-durability, outbox-failure, widget-handover, cache-persister, server-clock, sync-pull, turn-recovery, recurring-pay, export-file, api-errors, time-zone-use, widget-layout}.test.ts; apps/api/test/{recurring, assistant-money-drafts, assistant-actions, budgets, sessions, time}.test.ts; packages/shared/test/quick-add.test.ts

Area	Files
Third party (read only)	node_modules/expo-sharing 57.0.22 Android SharingModule.kt
Docs / raw	docs/AUDIT_PROMPT.md, PRODUCT.md, README.md, docs/audit/2026-10-07-r3/QA_Report.html, docs/audit/2026-10-07-r4/_raw/*